Course Code: CSA1741 **Course Name: ARTIFICIAL INTELLIGENCE**

Slot: B

ASSIGNMENT 2 – CO2

Nature of Assignment: Individual **Duration: 1 Week** **Total Marks: 50**

Student Name: Naresh Kanna. C . M **Reg. No: 192412093**

CO2 Coverage: Informed search strategies – Greedy best-first search – A* search – Heuristic functions – Local search algorithms and optimization problems – Online search agent – Constraint satisfaction problems – Game playing: adversarial search, minimax algorithm, alpha-beta pruning, evaluation functions.

TASK 1 – Greedy Best-First and A* Under Different Map Conditions (6 Marks)

Problem Context – Autonomous Campus Shuttle

A campus shuttle plans routes on grid maps of the campus. Obstacles (construction zones, closed roads) are placed in different patterns.

The development team is considering:

Uniform-Cost Search | Greedy Best-First Search | A* Search

However, the maps may be open, cluttered, maze-like or contain dead-end traps.

Your Challenge

Investigate whether one of the three algorithms should always be preferred or whether the appropriate choice depends on the characteristics of the map.

Before implementation:

- Predict how the three algorithms will behave under different map conditions
- Explain the algorithmic reason for your prediction
- Design your own admissible heuristic and justify its admissibility using a relaxed version of the problem

Design two deliberately contrasting maps and one additional edge-case map of your own that will expose meaningful behavioural differences among the algorithms.

Implement the three algorithms and record appropriate measures such as:

- Nodes expanded and nodes generated
- Maximum frontier size
- Path cost and whether the path is optimal
- Execution time, where useful

Compare your prediction with the observed behaviour, with code and outputs as proof.

Design Decision

The operations team reports that approximately 95% of shuttle trips are on open roads with only occasional detours around obstacles. Determine which algorithm you would recommend and defend your decision.

Evidence Expected: Reasoned prediction, a justified heuristic, three deliberately designed maps, implementation of the three algorithms, metric tables, comparison of predicted and observed behaviour and an evidence-based recommendation.

TASK 2 – When Is an Online Search Agent an Appropriate Engineering Choice? (4 Marks)

Problem Context – Exploration Rover in an Unmapped Facility

A rover must find a charging dock inside a facility whose map is not available. It can sense only the cells next to it, and it learns the map as it moves. The rover uses an online depth-first search (and, optionally, LRTA*).

Your Challenge

Design experiments to investigate the online agent under:

- Targets at different distances from the start
- Unreachable targets
- Different maze sizes and dead-end densities

Record the number of moves, revisited cells and the competitive ratio (cost travelled ÷ optimal cost with full map knowledge).

Then answer:

Under what operating conditions is an online search agent a reasonable engineering choice, and when would you stop recommending it?

Consider both the size of the environment and the number of repeated missions in the same facility (for example, learning across trials with LRTA*).

Changed Condition

The organisation now deploys 20 rovers across 20 facilities, and the maps remain unchanged for months. Re-evaluate your original recommendation.

Evidence Expected: Designed experiments, move-count and competitive-ratio evidence, analysis of successful and unsuccessful searches, an evidence-based operating boundary and a justified response to the changed condition.

TASK 3 – Engineer the Behaviour of Alpha-Beta Pruning (6 Marks)

Problem Context – Game Engine for a Board Game

A game company uses minimax with alpha-beta pruning for its two-player board game engine, and needs to understand how much work the pruning really saves.

Your Challenge

Instead of merely implementing the algorithm, investigate how the structure of the game tree influences computational effort.

Design:

1. One game tree in which alpha-beta pruning performs relatively little work (many cutoffs)
2. One game tree deliberately engineered so that almost no cutoffs occur
3. One additional test condition of your own choice (for example, random move order or a tree with equal leaf values)

Before executing each test:

- Predict the expected number of nodes visited
- Explain the algorithmic reason for your prediction

Implement minimax and alpha-beta and record the number of nodes visited and the number of cutoffs.

Inquiry

Compare prediction with experimental result. Explain:

- Why one tree requires substantially more work than another
- Which characteristics of move ordering create or prevent cutoffs
- What this reveals about evaluating an algorithm using only the branching factor and depth

Evidence Expected: Correct implementation, three designed trees, predictions before execution, node and cutoff counts, and a reasoned explanation of the differences.

TASK 4 – Minimax and the Cost of Examining Every Possibility (6 Marks)

Problem Context – Turn-Based Strategy Game

A game AI must choose its next move by looking ahead in a game with an average of b legal moves per turn.

Your Challenge

Design a depth-limited minimax solution with an evaluation function of your own.

Before implementation, determine:

- How many positions are examined for lookahead depth d
- How the evaluation function is used when the search is cut off
- How the number of terminal evaluations changes with d

Derive the number of nodes mathematically. Implement your solution and test it using at least three meaningfully different board sizes or depths.

Record:

- Number of nodes expanded and evaluation-function calls
- Execution time, where useful
- Quality of the move chosen (for example, win rate over 20 games against a random player)

Compare the theoretical growth with experimental behaviour.

Scalability Inquiry

Define an acceptable response time for your experimental environment. Using your evidence, determine approximately when plain minimax becomes impractical. Identify where the inefficiency originates and describe what a redesigned strategy would need to avoid or reduce. You are not required to implement the redesign in this task.

Evidence Expected: Algorithm design, mathematical node-count derivation, implementation, experimental growth evidence, a defined acceptability criterion and a reasoned identification of the limitation.

TASK 5 – Derive a Constraint-Propagation Algorithm from a Principle (7 Marks)

Problem Context – University Exam Timetabling

Exams that share students cannot be placed in the same slot. Each exam (variable) has a set of possible slots (domain).

You are required to develop a constraint-propagation solution.

Design Starting Point

You are given only the following principle:

A value may be removed from a variable's domain if some neighboring variable has no value that is compatible with it.

You are not provided with the complete algorithm.

Your Challenge

Convert this principle into a computational procedure. Determine:

- How the constraints are represented
- How a value without support is detected
- Which other pairs must be re-examined after a value is removed
- What condition indicates that the problem has no solution
- How the propagation is combined with backtracking search

Develop your own pseudocode before implementation.

Test Your Design

Test the algorithm using:

- A timetable instance in which propagation solves or detects failure without any search
- A second instance designed by you to challenge your algorithm

Visualise the shrinking domains. Compare backtracking with and without your propagation on nodes visited, backtracks, constraint checks and domain wipe-outs, and analyse how the work grows as the number of exams increases.

Correctness Inquiry

Explain:

Why does removing an unsupported value never remove a value that belongs to a solution?

Your explanation should be based on reasoning rather than program output.

Evidence Expected: A student-derived propagation rule, pseudocode, implementation, designed test cases, computational analysis, domain-reduction visualisation and a reasoned correctness argument.

TASK 6 – Exhaustive Search, Local Search and Combinatorial Growth (8 Marks)

Problem Context – Delivery Tour Planning

A courier must visit n customers exactly once and return to the depot, minimising the total travel distance.

Your Challenge

Design an exhaustive-search strategy. Determine how candidate tours will be represented, generated, evaluated and how the best tour will be retained. Do not begin from supplied pseudocode.

Correctness Check: Create a small instance whose tours can be independently verified and use it to establish that your algorithm finds the true optimum.

Growth Investigation

Before experimentation, derive/predict how the number of candidate tours grows with n. Use at least three problem sizes to validate your prediction experimentally.

Local Search Alternative

Design and implement a local-search method (for example, hill climbing with 2-opt moves with random restarts, or simulated annealing). For sizes where the exhaustive optimum is known, record the percentage by which your local-search tour exceeds the optimum, and the time taken, over at least 20 runs.

Inquiry

Explain why adding a small number of customers produces a dramatic increase in computational effort, relating your explanation to the solution space. Explain why local search can fail to reach the optimum (local minima, plateaus).

Deployment Decision

The planner must produce a plan within a fixed response time selected and justified by you. Estimate the largest n for which you would deploy exhaustive search, and the point at which you would switch to local search.

Evidence Expected: Student-designed exhaustive enumeration, correctness verification, growth prediction and validation, local-search quality and time evidence, and an evidence-based deployment boundary.

DAY-4 DESIGN CHALLENGE

TASK 7 – Diagnose, Redesign and Validate (13 Marks)

Review the evidence you generated while working on Tasks 1–6. Select **ONE** solution that demonstrates a significant computational limitation as its problem size or operating requirement increases.

You are not permitted simply to replace the solution with a more advanced textbook algorithm and state that its complexity is better. Understand the computational problem first and then develop an appropriate redesign.

A. Diagnose the Computational Bottleneck – 2 Marks

Using your own evidence, identify where most computational effort is spent, which operation grows significantly, whether work is unnecessarily repeated and which characteristic of the original algorithm causes the limitation.

B. Define the Design Objective – 2 Marks

Before proposing a new solution, state what your redesign must achieve (for example, reduce nodes expanded, exploit move ordering, reuse previously computed information, reduce constraint checks). Specify a measurable objective.

C. Design the Revised Computational Strategy – 4 Marks

Explain what you have changed, what property your new design exploits, which operations are avoided, whether preprocessing or extra memory is introduced and what new trade-offs are created. Provide pseudocode or a design representation. Marks are based on the quality of the reasoning, not on naming a known algorithm.

D. Predict Before Implementing – 1 Mark

Predict how the redesigned solution should differ from the original and identify the performance measure in which you expect improvement.

E. Implement and Validate the Redesign – 2 Marks

Implement a prototype and compare the original and redesigned solutions on the same or equivalent test conditions using nodes expanded, constraint checks, evaluation calls, path quality or other suitable measures, with code and output as proof.

F. Defend the Final Design – 2 Marks

Evaluate whether the redesign achieved its objective: whether the prediction was supported, the magnitude of improvement, new costs, and situations in which the original may still be preferable. A redesign that does not outperform the original is not automatically unsuccessful if you correctly analyse why.

MARK DISTRIBUTION – ASSIGNMENT 2

Task	Assessed Area	Marks
Task 1	Uniform-cost, Greedy and A* search	6
Task 2	Online search agent	4
Task 3	Alpha-beta pruning	6
Task 4	Minimax and evaluation functions	6
Task 5	Constraint propagation and CSP	7
Task 6	Exhaustive and local search, optimisation	8
Task 7	Day-4 Design Challenge – Diagnose, Redesign & Validate	13
	TOTAL	**50**

Submission Requirements

Submit:

- Your assignment report containing the required evidence for Tasks 1–7
- Source code for all implemented solutions
- Raw experimental data used to produce your results
- Tables/graphs used in your analysis, where applicable

Operation counts (nodes expanded, constraint checks, evaluation calls) should be treated as primary experimental evidence. Execution time may be reported as additional evidence but should not be the sole basis of your conclusions.

Assessment Verification

Students may be asked to explain any part of their algorithmic design, source code, test-data design, experimental results, predictions, analysis or design challenge during a brief viva. The student should be able to explain and defend the evidence submitted as their own work.

Overall Assessment Expectation

This assignment does not primarily assess your ability to reproduce textbook algorithms. It assesses your ability to:

Predict → Design → Implement → Measure → Analyse → Evaluate → Diagnose → Redesign → Validate

Your conclusions and design decisions must be supported by your own mathematical reasoning and experimental evidence.

CSA1741 Assignment 2 – Informed Search, Games, CSP and Optimisation

Nov 2, 2026 · @Saala

All numbers below were produced by the seven Python programs listed at the end; the Day-4 redesign (Task 7) turns the Task 6 exhaustive tour search into a branch-and-bound that expands about 300 times fewer nodes at 10 customers.

Task 1 – Greedy Best-First and A* under Different Map Conditions

1.1 Heuristic and relaxed problem

Grid, 4-connected moves, step cost 1. **h(n) = |row(n) − row(goal)| + |col(n) − col(goal)| (Manhattan distance).**

- *Relaxed problem:* the shuttle may drive through construction zones (all obstacles removed). In the relaxed problem every move is still one step and the cheapest route is exactly the Manhattan distance.
- *Admissible:* the real problem only adds constraints, so the real cost can never be below the relaxed cost: h(n) ⩽ h*(n).
- *Consistent:* a single move changes Manhattan distance by exactly 1, so h(n) ⩽ c(n,n′) + h(n′) with c = 1. Hence A* with a closed set never needs to reopen a node and returns an optimal path.

1.2 Prediction before implementation

Map	Uniform-cost (UCS)	Greedy best-first	A*
A: open (5% obstacles)	Expands every cell with g < C*, a diamond of radius C*: roughly half to most of the 3,600 cells. Optimal.	Heads straight for the goal: expansions ≈ path length (~120). Path near-optimal.	Expansions close to path length, a few hundred at most. Optimal.

Map	Uniform-cost (UCS)	Greedy best-first	A*
B: dead-end cup (start inside a cup whose mouth faces away from the goal)	Expands almost everything, optimal.	Falls into the trap: expands the cup interior, path not optimal.	Optimal, but also must fill the cup because f = g+h stays below C* there; more than greedy.
C: perfect maze	Expands the explored tree of the maze.	Heuristic points through walls, so no real guidance: similar to UCS.	h is almost uninformative: similar to UCS (>70% of UCS expansions).

Algorithmic reasons. UCS orders by g only, so it has no sense of direction. Greedy orders by h only: it is fast when h-descent matches the real route and is misled (and loses optimality) when the route must first move away from the goal. A* orders by g + h, so it keeps optimality (admissible h) and uses h to cut off cells whose g + h exceeds C*; in a maze h is far below the true remaining cost, so it prunes little.

1.3 Designed maps (60×60; the maze is 61×61)

- **A, open:** 5% random obstacles, start (0,0), goal (59,59).
- **B, dead-end trap:** no random obstacles; a wall from (10,40) to (50,40) with arms along rows 10 and 50 from column 20 to 40; start (30,30) inside the cup, goal (30,45) just behind the back wall. The only way out is through the mouth at column 20.
- **C, edge case (maze):** perfect maze made by recursive backtracking (seed 3); start (1,1), goal (59,59); exactly one path.
- **D (extra, cluttered):** 30% random obstacles.

Tie-breaking in the code: smaller h first (then insertion order); goal test at expansion for all three; metrics counted as: *expanded* = nodes popped and expanded; *generated* = nodes pushed; *max frontier* = largest heap size; *time* = mean of 5 runs.

1.4 Results (code `task1_search.py`)

Map	Algorithm	Expanded	Generated	Max frontier	Path cost	Optimal	Time (ms)
A open	UCS	3,398	3,398	61	118	yes	12.0
A open	Greedy	120	252	133	118	yes	0.58
A open	A*	120	252	133	118	yes	0.55

Map	Algorithm	Expanded	Generated	Max frontier	Path cost	Optimal	Time (ms)
B cup trap	UCS	3,286	3,319	59	79	yes	11.7
B cup trap	Greedy	1,268	1,358	91	79	yes	2.59
B cup trap	A*	1,268	1,358	91	79	yes	2.48
C maze	UCS	1,318	1,321	8	636	yes	2.2
C maze	Greedy	782	797	16	636	yes	1.4
C maze	A*	1,151	1,157	8	636	yes	2.0
D cluttered 30%	UCS	2,428	2,428	50	122	yes	7.3
D cluttered 30%	Greedy	344	448	105	192	**no (+57%)**	0.86
D cluttered 30%	A*	364	525	103	122	yes	0.97

The 95% case, averaged over 50 random open maps (5% obstacles, corner to corner):

Algorithm	Mean expanded	Mean path cost	Optimal in	Mean max frontier
UCS	3,419	118.0	100%	62
Greedy	126	123.7 (+4.8%)	4%	144
A*	204	118.0	100%	209

1.5 Prediction versus observation

- **Open map: mostly confirmed.** Greedy and A* both expanded 120 cells, exactly the path length; my guess of "a few hundred" for A* was too pessimistic because the tie-break on smaller h makes A* follow the straight-line bias. UCS expanded 3,398 cells, more than I predicted ("roughly half"): with C* = 118 the diamond of radius 118 covers nearly the whole 60×60 grid.
- **Cup trap: prediction partly wrong.** Greedy and A* expanded the same 1,268 cells and both found the optimal path. The reason is that every route out of the cup has to pass through the same mouth, so there is only one way out, and the cost of filling the cup is paid by both. The trap costs greedy optimality only when two routes of different length exist, which is what the cluttered map D (and the 96% non-optimal greedy paths on the 50 open maps) shows.
- **Maze: confirmed in the main**, A* ≈ UCS (1,151 vs 1,318 = 87%), as predicted. Greedy was better than predicted (782), because in a maze the dead-end branches are short and the manhattan order explores them cheaply. All three are optimal because the path is

unique.

1.6 Design decision (95% open-road trips)

Recommendation: A* with the Manhattan heuristic. On open maps it expands 17× fewer nodes than UCS (204 vs 3,419) at the same optimal cost, while greedy saves only another 38% of expansions (126 vs 204) but returns a non-optimal path in 96% of the trips (average +4.8% travel) and +57% on a cluttered map. Because the heuristic is consistent, A* never does worse than UCS in expansions (it equals UCS when h = 0). Costs: A* keeps a larger frontier (209 vs 62 entries on average) and a heuristic evaluation per node, which are trivial on a campus map. In the 5% of mazes or dense obstacle fields A* degrades gracefully to roughly UCS, and it stays optimal. The choice therefore does depend on the map for greedy (unsafe) and UCS (wasteful), but one algorithm, A*, is the best default for this fleet.

Task 2 – When Is an Online Search Agent a Reasonable Choice?

2.1 Experimental design (code `task2_online.py`)

- **Environment:** grid mazes from a recursive-backtracker (perfect maze, many dead ends). A "braid" parameter opens a fraction of dead ends into loops, which lowers the dead-end density (measured as dead-end cells ÷ free cells). Sizes 21×21, 41×41, 61×61 (199, 799, 1,799 free cells); braid 0, 0.5, 0.9; 10 mazes (seeds 0–9) per configuration.
- **Agent:** senses only the four neighbouring cells. *Online DFS* remembers visited cells and the path back, and backtracks when stuck. *LRTA** keeps a table H of learned costs (initialised with Manhattan distance to the target), moves to the neighbour minimising 1 + H and updates H of the cell it leaves; the table persists across trials, so 10 trials were run in each maze.
- **Targets:** at 25%, 50% and 100% of the farthest distance from the start (1,1); and an *unreachable* target (a free cell sealed in by walls).
- **Metrics:** moves, revisits (moves into an already visited cell, i.e. backtracking), and the competitive ratio = moves ÷ optimal cost (BFS on the full map). Move counts are the primary evidence.

2.2 Results

Effect of maze size (perfect maze, farthest target; mean of 10 mazes):

Size	Free cells	Optimal cost	DFS moves	DFS revisits	DFS ratio	LRTA* ratio, trial 1	LRTA* ratio, trial 10
21×21	199	147.8	215	33.6	1.48	10.4	3.3
41×41	799	439.2	870	215.6	1.98	23.4	8.7
61×61	1,799	945.6	1,749	401.8	1.90	32.6	8.4

Effect of target distance (41×41 perfect maze):

Target at	Optimal cost	DFS moves	DFS revisits	DFS ratio	LRTA* trial 1	LRTA* trial 10
25%	109.8	271	81	2.62	4.0	1.8
50%	219.6	299	40	1.36	9.5	3.5
100%	439.2	870	216	1.98	23.4	8.7

Effect of dead-end density (41×41, farthest target):

Braid	Dead-end density	DFS moves	DFS revisits	DFS ratio	LRTA* trial 1	LRTA* trial 10
0	5%	870	216	1.98	23.4	8.7
0.5	3%	593	176	3.10	23.9	6.9
0.9	0.0%	418	120	3.46	12.4	3.9

Near targets in loop-rich mazes are the worst case for DFS: in the 61×61 maze with braid 0.9 and a target at 25% (optimal cost 46.5), DFS needed 1,290 moves, a ratio of **28.7**. DFS has no sense of direction, so a short shortcut is found only after it wanders.

Learning across trials (LRTA*, 41×41, braid 0, farthest target, mean of 10 mazes): ratio per trial 23.4, 10.1, 9.2, 9.9, 6.8, 8.5, 7.6, 7.1, 6.1, 8.7. It falls fast at first but is still above 6 after ten trials, so it does not approach 1 in a practical number of missions.

Unreachable target: online DFS always terminated and reported failure, with moves = 2 × (free cells − 1) (396 moves for 199 cells, 1,596 for 799, 3,596 for 1,799): it explores every reachable cell once and backs out. LRTA* never terminated on its own: all 25 of 25 runs reached the move cap (300 × free cells). Online DFS is complete; LRTA* is not when the target can be unreachable.

2.3 Analysis

- **Successful searches.** In a perfect maze the competitive ratio of online DFS stays between 1.1 and 4.8 (about 2 for the farthest target) regardless of size, because it never explores more than every corridor twice (⩽ 2 × cells − 1 moves) and the unique route is a large share of the maze. In loop-rich mazes the ratio rises to 3 to 29, since the optimal path is short but DFS commits to the first branch.
- **Unsuccessful searches.** The cost of failing is the whole reachable map, 2(cells − 1) moves, with no way to know earlier. LRTA* wastes unbounded effort.
- **Size.** Moves of online DFS grow linearly with free cells (215, 870, 1,749 for 199, 799, 1,799 cells) and memory is O(cells) (visited set). The ratio did not blow up with size in perfect mazes.

2.4 Operating boundary

An online agent is a reasonable engineering choice when **all** of the following hold: (1) no map exists and building one first costs more than the extra travel (a one-off or rarely repeated mission); (2) the facility is small or tree-like, so that 2(cells − 1) moves is an acceptable worst case (up to a few thousand cells in my runs, with ratio ⩽ 3); (3) the target can be recognised on sensing; (4) the agent has memory for O(cells) visited cells. Stop recommending it when: the same facility is visited repeatedly (every mission pays the exploration again, with ratios of 2 to 29 each time); the environment is large or loop-rich with near targets (ratio up to 29); or unreachable targets must be detected cheaply (cost = the whole map). LRTA* learning across trials is not a rescue: after 10 trials the ratio was still 3.3 to 8.7.

2.5 Changed condition: 20 rovers, 20 facilities, maps unchanged for months

The assumption "map unavailable" no longer holds after the first visit, so I **change the recommendation**: use online DFS only as a *mapping run* once per facility, store the map it discovers (a grid of at most a few thousand cells, a few KB) and plan every later mission offline with A* (ratio 1.0, Task 1). Evidence: in the 41×41 maze one online mission costs about 870 moves against 439 for the optimal route, so each repeated mission saves about 431 moves. A complete mapping run costs at most 2 × (799 − 1) = 1,596 moves, so the investment pays back after about 4 missions per facility (1,596 ÷ 431 ≈ 3.7), and sooner when the 20 rovers share the facility map or when the first mission's own exploration is reused. Over months of operation the online agent is only appropriate as a fallback when the stored map no longer matches the sensed walls (a blocked corridor), triggering re-mapping of that region.

Task 3 – Engineering the Behaviour of Alpha-Beta Pruning

3.1 Test design (code `task3_alphabeta.py`)

Uniform game trees with branching factor b = 4 and depth d = 8 (65,536 leaves), random integer leaf values 0–1,000, seed 1. Trees are stored explicitly, so move order is an explicit property of the tree. "Nodes visited" counts every node entered (internal nodes and leaves). A cutoff is counted each time the test α ⩾ β stops a loop early.

1. **Many cutoffs (best-first ordering):** at every node the children are sorted by their true minimax value, best move first (maximising nodes highest first, minimising nodes lowest first).
2. **Almost no cutoffs (worst-first ordering):** the same tree with every node's children sorted worst move first.
3. **Own test A: random move order:** the unsorted random tree. **Own test B: equal leaf values:** all 65,536 leaves = 7.

All four trees give the same minimax value as plain minimax (checked by assertion).

3.2 Predictions (made before running) and observations

- *Minimax:* visits every node, (b^(d+1) − 1)/(b − 1) = 87,381.
- *Best-first:* only the minimal (critical) tree is examined. Using Knuth–Moore node types: a PV node expands all its children (one PV child, b−1 cut-nodes), a cut-node expands exactly one child (an all-node), an all-node expands all b children (cut-nodes). Counting level by level for b = 4, d = 8 gives **1,098 nodes** (≈ 2·b^(d/2) leaves).
- *Worst-first:* a cutoff can occur only when the move examined first is not the best one; here the best move comes last everywhere, so I predicted essentially no cutoffs, close to 87,381 nodes.
- *Random order:* by the Pearl/Knuth–Moore result the effort is about b^(3d/4); I predicted ≈ 8,000 nodes.
- *Equal leaves:* every comparison is a tie and the test is α ⩾ β, so a node can be cut off as soon as one child has returned the common value; I predicted **at most** the best-first count.

Tree	Predicted nodes	Observed nodes	Cutoffs	Observed / minimax
Minimax (any order)	87,381	87,381	0	100%

Tree	Predicted nodes	Observed nodes	Cutoffs	Observed / minimax
Alpha-beta, best-first	1,098	1,098	417	1.3%
Alpha-beta, worst-first	≈ 87,381	66,123	12,381	75.7%
Alpha-beta, random order	≈ 8,000	11,191	2,735	12.8%
Alpha-beta, equal leaves	⩽ 1,098	1,098	417	1.3%

Depth sweep (b = 4, nodes visited by alpha-beta) and branching sweep (d = 6):

Depth d	Minimax	Best-first	Random	Worst-first
2	21	12	17	21
4	341	62	169	322
6	5,461	268	1,297	4,725
8	87,381	1,098	11,191	66,123

Branching b	Minimax	Best-first	Random	Worst-first
2	127	44	72	124
3	1,093	125	300	996
4	5,461	268	1,297	4,725
6	55,987	812	8,034	49,173
8	299,593	1,820	29,148	252,756

3.3 Prediction versus experiment

- **Best-first matched exactly** (1,098 = 1,098 at every depth and branching tested), which confirms the critical-tree analysis. Equal leaves reached the same count rather than fewer: ties create cutoffs at cut-nodes only, and the all-nodes still need every child, so the minimal tree is also the floor here.

- **Random order** came out 40% above my estimate (11,191 vs 8,000): my constant (1.5) was a guess; the growth rate is right: the node count grows by 8.6 when depth grows by 2 (1,297 → 11,191), close to b^(3/2) = 8 predicted for exponent 3d/4, far below minimax's 16.
- **Worst-first was wrong in detail:** I predicted ≈ 100% and saw 75.7%. Cutoffs still occurred (12,381) but removed only 24% of the work. A likely reason, which I did not test separately, is that near the leaves the bounds inherited from higher up are already tight enough for the first or second child to trigger α ≥ β even with bad ordering, while the high-level cutoffs that remove large subtrees are the ones that disappear.

3.4 Explanation

- **Why one tree needs far more work.** Alpha-beta does not make the search cheaper by itself; it only skips children that cannot change the decision. Whether it can skip them depends on how early a *good bound* (α at max nodes, β at min nodes) becomes available. With best-first order the first child already supplies the true value, so every other child is refuted by examining one reply: effort ≈ b^(d/2). With worst-first order the early bounds are weak, each later sibling improves them a little, and few siblings can be cut: effort ≈ b^d.
- **Which characteristics create cutoffs.** (1) Best move first at max nodes, so α rises early; (2) at min nodes, the refuting reply (best for the opponent) first, so a single reply cuts the node; (3) ties (equal values) help only at cut-nodes. Characteristics that prevent cutoffs: ascending order for the maximiser, descending for the minimiser, and bounds that start at ±∞.
- **What this says about b and d alone.** The same b = 4, d = 8 tree is searched with 1,098, 11,191 or 66,123 nodes depending only on the order of children: a factor of 60 between two orderings of the *same* tree. Branching factor and depth fix only the upper limit b^d and the best case b^(d/2); the real effort lies between them and is set by move ordering. Evaluating an engine from b and d alone is therefore misleading; it must be measured with the engine's own move ordering (e.g. captures first, or the previous iteration's best move).

Task 4 – Minimax and the Cost of Examining Every Possibility

4.1 Design (code `task4_minimax.py`)

- **Game:** Connect-4 (R rows × C columns, win with four in a row), the average number of legal moves b is the number of non-full columns (7 on the standard 6×7 board).
- **Search:** depth-limited minimax. A position is *terminal* if the previous move won or the board is full (value ±1,000,000 or 0). At depth 0 and non-terminal, the evaluation function is called instead of searching on.
- **Evaluation function (my own):** from the point of view of the side being evaluated, 3 points per own piece in the centre column minus 3 per opponent piece there, plus for every one of the 69 four-cell windows that contains only one colour: +1, +10, +100 for one, two, three pieces (negative for the opponent). Windows containing both colours score 0.

4.2 Mathematical node count

Let every non-terminal position have b children. The root is at depth 0; positions at depth i number b^i.

```
\text{expanded (internal) nodes} = \sum_{i=0}^{d-1} b^i = \frac{b^d-1}{b-1},
\qquad \text{evaluation calls (leaves at depth } d) = b^d
```

```
\text{total positions examined} = \sum_{i=0}^{d} b^i = \frac{b^{d+1}-1}{b-1}
= \Theta(b^d)
```

So each extra ply multiplies the evaluations by b (×7), and the evaluation function is called at every one of the b^d frontier positions: it is the cost-dominant operation (about 85.7% of all positions for b = 7). Terminal positions and full columns make the real counts smaller than these bounds.

4.3 Experiments

Growth with depth (empty 6×7 board, program to move, b = 7):

Depth d	Predicted internal (b^d−1)/(b−1)	Observed expanded	Predicted b^d	Observed evaluation calls	Time (s)
1	1	1	7	7	0.0002
2	8	8	49	49	0.0015

Depth d	Predicted internal (b^d−1)/(b−1)	Observed expanded	Predicted b^d	Observed evaluation calls	Time (s)
3	57	57	343	343	0.010
4	400	400	2,401	2,401	0.045
5	2,801	2,801	16,807	16,807	0.34
6	19,608	19,608	117,649	117,649	2.17

The match is exact because no position within six plies of the empty board is terminal. In a mid-game position (10 random plies) the same search shows the effect of terminal positions and full columns: at d = 6 only 61,391 evaluation calls and 11,871 expanded nodes (about half of the bound), 1.23 s.

Different board widths (6 rows, depth 4, empty board): C = 5, 7, 9 columns, predicted b^4 = 625; 2,401; 6,561 evaluations, observed 625; 2,401; 6,561; expanded 156; 400; 820 (predicted 156; 400; 820), time 8 ms; 48 ms; 163 ms.

Quality of the move chosen (20 games per row, first move alternating, results from the program's side):

Opponent	Depth	Wins	Draws	Losses
Random player	1	20	0	0
Random player	2	20	0	0
Random player	3	20	0	0
Random player	4	20	0	0
Random player	5	20	0	0
Depth-1 player (2 random opening plies)	2	17	1	2
Depth-1 player	3	18	0	2
Depth-1 player	4	15	1	4
Depth-2 player	4	13	1	6

A random opponent is too weak to separate the depths (100% win rate at every depth, even depth 1), so I added head-to-head games. Depth 2 and 3 clearly beat depth 1 (85% to 90%); depth 4 did not score higher than depth 3 in 20 games (75% against depth 1), a

difference that is within the noise of 20 games, and my evaluation function may be the limiting factor beyond depth 3. Total effort for the 20 games vs random at depth 5 was 1,870,207 evaluation calls and 38 s, against 708 calls and 0.017 s at depth 1.

4.4 Theory versus experiment

Growth agrees with b^d exactly where no terminal positions occur and falls to about half of it in mid-game (61,391 against 117,649 at d = 6); time grows by 6.4 from d = 5 to d = 6 (0.34 s → 2.17 s), a little less than the ×7 node growth because shallow nodes cost less to expand than evaluation calls.

4.5 Scalability inquiry

- **Acceptable response time:** 1 second per move for a turn-based game on this machine (Python).
- **When plain minimax becomes impractical:** depth 5 takes 0.34 s on the empty board and 0.19 s mid-game: acceptable. Depth 6 takes 1.2 to 2.2 s: over the limit. Depth 7 would take about 7 × 2.17 ≈ 15 s (predicted, not run). So plain minimax is practical up to **depth 5 for b = 7**; with b = 9 (width 9) the limit drops to depth 4 (163 ms at d = 4, ≈ 1.5 s predicted at d = 5), and each added column costs more than a factor 2 at depth 5.
- **Where the inefficiency comes from:** (1) every one of the b^d frontier positions is evaluated, though most are obviously bad moves and cannot affect the choice; (2) the same position is reached by different move orders (transpositions) and re-searched; (3) the search depth is fixed instead of adapting to the time left.
- **What a redesign must do (not implemented here):** skip branches that cannot change the decision (bound-based pruning with good move ordering, centre columns first, Task 3), remember positions already evaluated (a transposition table), and search by iterative deepening against a time budget so the best move so far is always available. A redesign succeeds if it reaches depth 7 within 1 s, which needs roughly a 15× reduction in positions examined.

Task 5 – Deriving a Constraint-Propagation Algorithm from a Principle

Principle given: *a value may be removed from a variable's domain if some neighbouring variable has no value compatible with it.*

5.1 From principle to procedure

- **Representation.** Variables = exams. Domain D(x) = set of allowed slots (unary restrictions such as "not on Monday" are removed from D(x) at the start). A constraint is an edge x–y in the clash graph (the exams share a student) meaning slot(x) ≠ slot(y). Each edge is stored as two directed arcs (x,y) and (y,x) in adjacency sets.
- **Detecting an unsupported value.** For an arc (x,y) and a value v ∈ D(x), scan D(y) for a value w compatible with v (w ≠ v). If none exists, v is unsupported. Each compatibility test is one *constraint check*.
- **What must be re-examined after a removal.** If D(x) lost a value, a neighbour z may have relied on that value, so every arc (z,x) pointing *into* x (except the arc (y,x) whose revision caused it) goes back on the queue. Arcs (x,·) need no re-check, since D(y) did not change.
- **No-solution condition.** If any domain becomes empty (a *domain wipe-out*), the problem has no solution (proved in 5.4).
- **Combination with backtracking (MAC).** Assign x := v by setting D(x) = {v} on a *copy* of the domains, then run the propagation starting with the arcs (z,x) of the neighbours of x. A wipe-out means the value fails: undo it by discarding the copy and try the next value; if none works, backtrack. Plain backtracking, for comparison, only checks the new value against already assigned neighbours.

5.2 Pseudocode (my derivation)

```
REVISE(x, y):                       # remove unsupported values of x w.r.t. y
  removed = false
  for v in D(x):
    if no w in D(y) with w != v:      # constraint checks counted here
      remove v from D(x); removed = true
  return removed

PROPAGATE(queue):                   # queue holds arcs (x, y)
  while queue not empty:
    (x, y) = pop(queue)
    if REVISE(x, y):
      if D(x) is empty: return FAILURE          # domain wipe-out
      for z in neighbours(x), z != y: push (z, x) if not already queued
  return OK

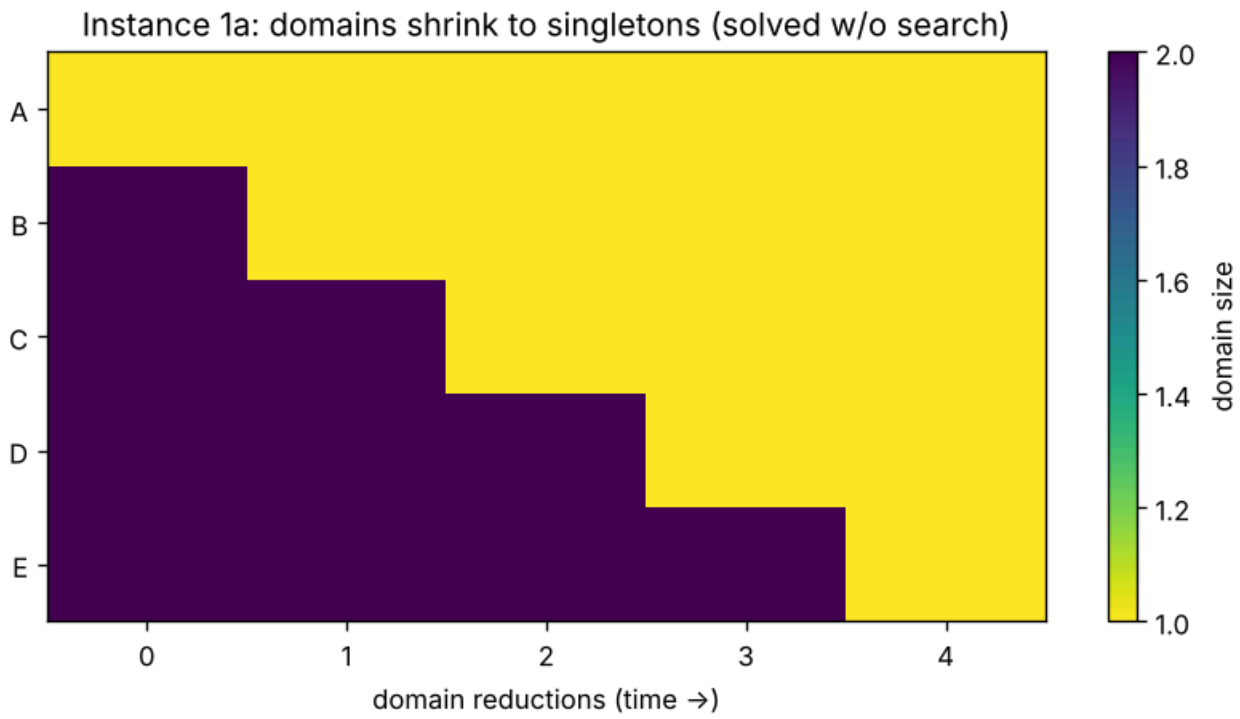
SOLVE(i, domains):                  # static order: exams with most clashes
first
```

```
  if i == n: return solution
  x = order[i]
  for v in domains[x]:
    d2 = copy(domains); d2[x] = {v}
    if PROPAGATE({(z, x) : z in neighbours(x)}) on d2 != FAILURE:
      if SOLVE(i + 1, d2) succeeds: return it
    count a backtrack
  return FAILURE
```

Before the search the whole problem is propagated once with all arcs in the queue; if that already fails, there is no solution and no search is run.

5.3 Test cases and results (code `task5_csp.py`)

Instance 1a: solved with no search. Exams A–E in a chain of clashes A–B–C–D–E, slots 1–3 with availability windows D(A)={1}, D(B)={1,2}, D(C)={2,3}, D(D)={2,3}, D(E)={1,2}. Propagation (15 constraint checks) ended with A=1, B=2, C=3, D=2, E=1, a complete valid timetable, because each removal made the next neighbour's domain a singleton.



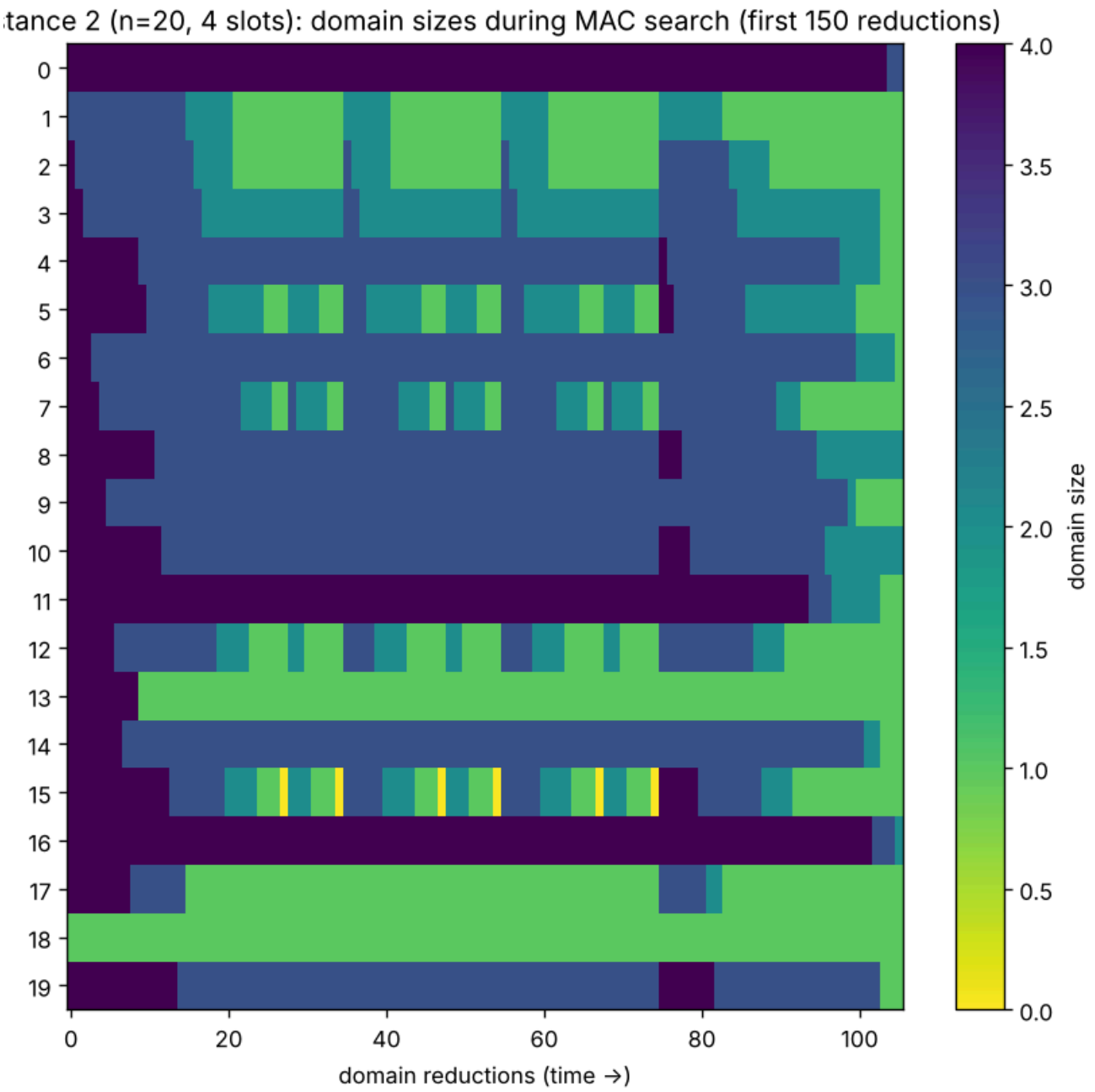
Instance 1b: failure detected with no search. Triangle F, G, H (all three share students), slots {1,2}, with F fixed to slot 1. Propagation made G={2} and then found H without support: H's domain became empty (1 wipe-out, 10 checks) so the instance was declared unsatisfiable without choosing any value.

Instance 1c: the limit of propagation. Four exams that all clash with each other and only 3 slots: no solution exists, but every value of every exam still has support, so propagation alone reports "consistent" (it cannot see the failure). Search is needed: plain backtracking 48 nodes, MAC 9 nodes with 6 wipe-outs.

Instance 2 (my challenge): random timetables. n exams, 4 slots, an average of 5.5 clashes per exam (random clash graph, 30 seeds per size, some instances unsatisfiable), same static variable order for both methods, cap of 2,000,000 nodes. Wherever plain backtracking finished, both methods agreed on solvable/unsolvable (n = 20 and 30 contain 3 and 2 unsatisfiable instances), and every solution was verified against all clashes.

n exams	BT nodes (mean)	BT nodes (median)	BT backtracks	BT checks	MAC nodes (mean)	MAC nodes (median)	MAC backtracks	MAC wipe-outs	MAC checks
20	1,594	56	388	4,021	24	20	6.2	3.8	1,868
30	6,077	78	1,503	15,600	59	30	31	20	4,694
40	2,927	115	709	7,255	43	40	3.3	2.1	2,806
50	32,379	233	8,066	85,254	50	50	0	0	3,272
60	119,892	865	29,939	308,540	60	60	0.2	0.2	3,977
80	358,522 (5 runs hit the cap)	2,663	89,596	956,141	290	80	210	111	14,023

(Plain backtracking never has a domain wipe-out because it does not shrink domains: 0 in every row.)



5.4 Growth with the number of exams

Plain backtracking is heavy-tailed: the median stays small (56 to 2,663 nodes) but the mean explodes (1,594 at n = 20 to 358,522 at n = 80, and 5 of the 30 instances at n = 80 did not finish within 2,000,000 nodes), because one early wrong choice can cause thrashing deep below. MAC's median is exactly n (no backtracking at all in most instances), its mean only 290 at n = 80, and it finished all 30 instances; its constraint checks grow roughly linearly (1,868 to 14,023) while backtracking's grow to 956,141. The price is the work per node: MAC used about 77 checks per node at n = 20 (1,868 ÷ 24) against about 2.5 for

backtracking (4,021 ÷ 1,594). So propagation wins when search is hard and loses a little on trivially easy instances, where its extra checks per node are not recovered. Operation counts, not time, are the evidence.

5.5 Correctness inquiry: why removing an unsupported value never loses a solution

Let S be the set of all complete solutions that respect the current domains. **Invariant:** every solution in S has slot(x) ∈ D(x) for every x. It holds at the start (the domains are the allowed slots).

Suppose value v is removed from D(x) because neighbour y has no w ∈ D(y) with (v,w) allowed. Take any solution σ in S with σ(x) = v. By the invariant σ(y) ∈ D(y). The constraint between x and y requires (σ(x), σ(y)) = (v, σ(y)) to be allowed. But no element of D(y) is allowed together with v, a contradiction. Hence no solution in S uses v at x: removing v removes no solution, and the invariant still holds after the removal. Each removal preserves the invariant, so by induction after any number of removals every solution is still contained in the domains.

Consequences. (1) A domain wipe-out means S is empty: a solution would have to give x a slot inside an empty set. (2) In the combined search, assigning x := v is just the same as removing every other value from D(x); the argument above applies unchanged, so MAC never discards a solution consistent with the assignments made so far. (3) The procedure terminates: each queue entry is added only after a removal, and there are at most Σ|D(x)| removals. (4) The converse is false: when propagation stops with non-empty domains the problem can still be unsolvable (instance 1c), so search is still required.

Task 6 – Exhaustive Search, Local Search and Combinatorial Growth

6.1 Exhaustive strategy (my design, code `task6_tsp.py`)

- **Representation:** a tour is a list that starts at the depot (city 0) followed by the order of the other n − 1 customers; the return edge to the depot is implicit. Fixing the depot as first city removes the n rotations of the same cycle.
- **Generation:** depth-first extension of a partial tour by each unvisited customer in turn, so every one of the (n − 1)! complete tours is produced exactly once and tours that share a prefix share its cost.

- **Evaluation:** a running cost is kept; when the tour is complete, the closing edge back to the depot is added.
- **Best tour retained:** a variable holds the cheapest cost and a copy of the path; it is replaced whenever a complete tour is cheaper.

6.2 Correctness check

1. **Known optimum:** 8 customers on a circle (labels shuffled): the optimal tour of points in convex position is the polygon, with cost equal to the perimeter $n \cdot 2R \sin(\pi/n) = 244.917397$. The exhaustive search returned 244.917397 after examining 5,040 = 7! tours.
2. **Independent oracle:** on 20 random 7-city instances the result equalled the minimum over `itertools.permutations` of the same instance in 20 of 20 cases.

6.3 Growth: prediction and validation

Prediction (before running). Fixing the depot leaves $n - 1$ customers in any order, so the number of tours is $(n - 1)!$, and adding one customer multiplies the work by n (from n to $n + 1$ customers: $n!/(n - 1)! = n$). (Directions double count each tour, so there are $(n - 1)!/2$ distinct tours; my search does not exploit this.) Internal nodes: $\Sigma_k (n - 1)!/(n - 1 - k)! \approx e \cdot (n - 1)!$.

n	Tours examined	(n−1)!	Nodes	Time (s)	µs per tour	Ratio to previous n (tours)
4	6	6	16	0.00002	3.2	
5	24	24	65	0.00004	1.7	×4.0
6	120	120	326	0.0002	1.6	×5.0
7	720	720	1,957	0.0013	1.9	×6.0
8	5,040	5,040	13,700	0.0085	1.7	×7.0
9	40,320	40,320	109,601	0.070	1.7	×8.0
10	362,880	362,880	986,410	0.50	1.4	×9.0

The tour counts match $(n - 1)!$ exactly for seven sizes, the node counts match $e \cdot (n-1)!$ (986,410 vs 986,410 at $n = 10$), and time grows by 7 to 8 per added customer once the run is long enough to measure.

6.4 Local search alternative (2-opt)

Method: start from a random tour; repeatedly look for a pair of edges (a,b), (c,d) such that replacing them by (a,c), (b,d) (reversing the segment between) shortens the tour; apply the first improving move; stop when none exists (a 2-opt local minimum). **Random restarts:** repeat from new random tours and keep the best. 20 runs on each of 3 random instances per size = 60 runs per row, compared with the exhaustive optimum of the same instance.

n	Variant	Mean excess over optimum	Worst run	Runs that reached the optimum	Mean time (ms)
8	2-opt, single start	0.35%	3.51%	90%	0.04
8	2-opt, 10 restarts	0.00%	0.00%	100%	0.14
9	2-opt, single start	0.55%	3.17%	72%	0.02
9	2-opt, 10 restarts	0.13%	1.31%	90%	0.17
10	2-opt, single start	0.44%	2.37%	72%	0.03
10	2-opt, 10 restarts	0.04%	0.53%	92%	0.25

Beyond the exhaustive range (no optimum known): 2-opt with 10 restarts takes 0.9 ms for n = 20, 6.9 ms for n = 50 and 39 ms for n = 100 (mean of 20 runs); for n = 50 the 20 runs gave costs from 516.4 to 530.9, so the answer varies by up to 2.8%.

6.5 Inquiry

Why a few more customers cost so much. The solution space is the set of all orderings of the customers: it has (n − 1)! members, so each added customer multiplies it by n, not adds to it. Going from 10 to 13 customers multiplies the number of tours by $10 \cdot 11 \cdot 12 = 1{,}320$; from 10 to 15 by $10 \cdot 11 \cdot 12 \cdot 13 \cdot 14 = 240{,}240$. Exhaustive search must visit all of them because without extra knowledge any tour might be the best, so the work follows the size of the space, and the space grows faster than any exponential.

Why local search can miss the optimum. 2-opt only compares a tour with its *neighbours* (tours reachable by one segment reversal). A **local minimum** is a tour with no cheaper neighbour although a better tour exists further away: single-start 2-opt ended in one in

10% (n = 8) to 28% (n = 9, 10) of runs, with up to 3.5% excess. **Plateaus** (several neighbouring tours of equal cost, common with equal or rounded distances) give the search no direction, so it stops or wanders; my first-improvement rule stops when no strictly cheaper neighbour exists. Random restarts help because each start descends into a different basin of attraction: ten restarts lifted the success rate from 72% to 90–92% at n = 9–10, but they give no guarantee.

6.6 Deployment decision

Response time chosen: 2 seconds (a dispatcher expects a plan on screen almost at once). Measured speed was 1.4 µs per tour at n = 10. For n = 10, 362,880 tours take 0.5 s (measured); n = 11 would need 3.6 million tours ≈ 5 s, over the limit. **Deploy exhaustive search up to n = 10 customers; switch to local search from n = 11.** Local search runs in milliseconds even at n = 100 and was within 0.00% to 0.13% of the optimum on average with 10 restarts (0.35% to 0.55% with a single start) where the optimum is known, though not guaranteed optimal. (The redesign in Task 7 moves the exact-search boundary to about n = 12 with a hard guarantee, and to about n = 16 on average.)

Task 7 – Day-4 Design Challenge: Diagnose, Redesign and Validate

Solution selected: the exhaustive courier-tour search of Task 6. Its evidence shows the sharpest limit: the response time of 2 s is reached at only n = 10 customers.

7.1 Diagnosing the bottleneck (2 marks)

- **Where the effort goes:** every node of the search tree is a partial tour; the program creates ≈ e·(n−1)! of them (986,410 at n = 10, matching e·9! exactly) and each complete tour is priced at about 1.4 µs. The operation that grows is *extending a partial tour by one more customer* (and the final tour evaluation).
- **Is work unnecessarily repeated or wasted?** Yes, in two ways. (1) The search keeps extending partial tours whose cost so far is already larger than a complete tour found earlier: such a partial tour can never become the best, yet all its (n − 1 − k)! completions are generated. (2) Nothing learned from earlier tours is used: the best cost seen so far is stored but never used to stop work.
- **Characteristic of the algorithm that causes it:** it is *blind*, i.e. it never asks whether a branch can still win. Its work is set by the size of the permutation space (Task 6.5), not by how good the best tour is.

7.2 Design objective (2 marks)

The redesigned solver must (a) return **exactly the same optimal cost** as the exhaustive search (verified on every test instance up to n = 10); (b) expand **at least 100 times fewer nodes** than exhaustive search at n = 10; (c) solve **n = 14 within the 2 s response time on average** (the exhaustive search would need about 2.4 hours); and (d) keep memory at O(n) for the stack plus O(n²) for preprocessed tables. The measures are nodes expanded (primary) and time (secondary).

7.3 Revised computational strategy (4 marks)

What is changed. Keep the depth-first enumeration of Task 6, but let every node decide whether it can still lead to a tour cheaper than the best known one.

Property exploited: edge costs are non-negative and every unvisited city must still be left exactly once, so a *lower bound* on the cost of completing a partial tour is available cheaply, and a good *upper bound* (the cost of an actual tour) is available from the 2-opt local search of Task 6.

1. **Upper bound from local search (preprocessing).** Run 2-opt with 5 restarts first; its tour cost becomes the initial `best`. (Cost: about 0.2 ms; reuses Task 6.)
2. **Partial-cost pruning.** If cost-so-far ⩾ best, cut the branch (all its completions are at least as expensive).
3. **Lower bound on the rest.** LB = the sum, over the current city and every unvisited city v, of the cheapest edge leaving v towards an unvisited city or the depot. It is admissible: each of those cities must be left once in any completion, and no edge leaving v costs less than its cheapest one. Prune when cost-so-far + LB ⩾ best.
4. **Nearest-first ordering.** Try the nearest unvisited city first, so that good tours (tight bounds) are found early.
5. **Preprocessing/memory:** sorted neighbour lists per city O(n² log n); the 2-opt tour; extra work per node O(n) for the lower bound. **New trade-offs:** each node costs several times more than before, the running time now depends on the instance (some layouts prune badly), and the worst case is still exponential.

Avoided operations: all completions of a pruned partial tour, i.e. whole subtrees of size (n − 1 − k)!.

```
BNB(D):
  best = cost(2-opt tour with restarts); bestTour = that tour          #
upper bound
  near[v] = other cities sorted by distance from v                      #
preprocessing
  EXTEND(path = [0], cost = 0):
    if all cities visited: total = cost + D[last][0]; update best if smaller;
return
    if cost + LB(path) >= best: return                                  #
lower-bound prune
    for v in near[last] not yet visited:                                #
nearest first
      if cost + D[last][v] >= best: continue                            #
partial-cost prune
      EXTEND(path + [v], cost + D[last][v])
  LB(path) = sum over v in {last} U unvisited of min D[v][w], w unvisited or
depot, w != v
```

7.4 Prediction before implementing (1 mark)

I predicted: (1) optimal cost identical to the exhaustive search; (2) nodes expanded at n = 10 reduced by two orders of magnitude or more (≥ 100×), and growth with n far below factorial; (3) the *time* improvement smaller than the node improvement, because each node is now several times more expensive. I made no prediction about which single ingredient would matter most; the ablation in 7.5 measures that. The measure in which improvement is expected: **nodes expanded**.

7.5 Implementation and validation (2 marks) (code `task7_bnb.py`)

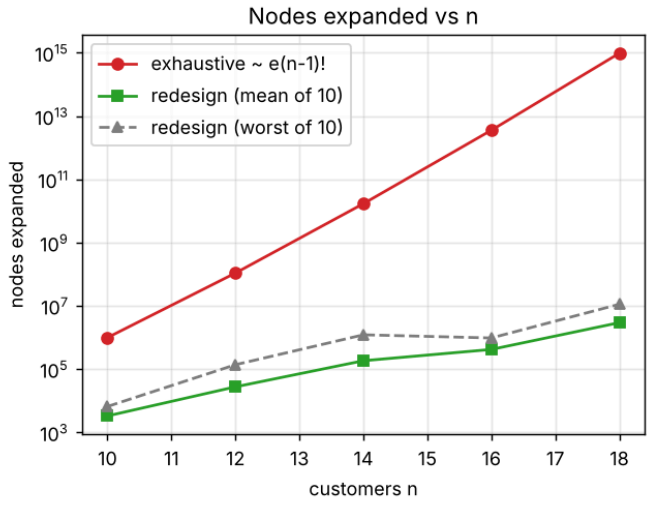
Same instances for all variants (random points in a 100×100 square, seeds 0–9 per size); optimal cost compared with the exhaustive search for every instance at n = 8 and 10 (0 mismatches in all 20).

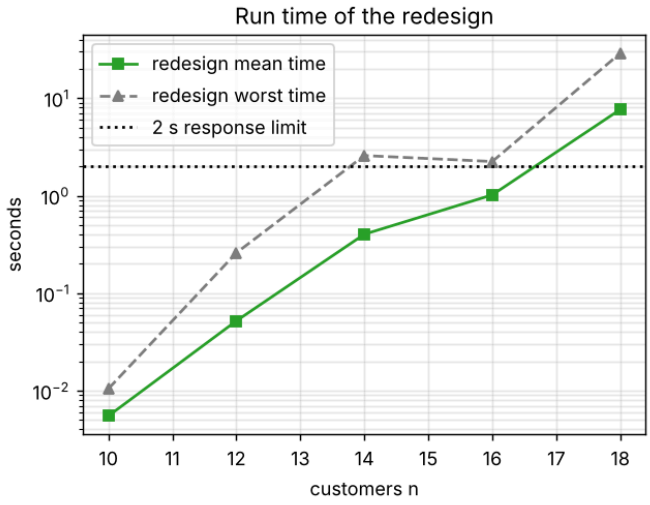
n	Variant	Mean nodes expanded	Reduction vs exhaustive	Mean time (ms)
8	Exhaustive (Task 6)	13,700	1×	8.5
8	A: partial-cost prune only	4,006	3.4×	2.7

n	Variant	Mean nodes expanded	Reduction vs exhaustive	Mean time (ms)
8	B: A + nearest-first order	3,460	4.0×	3.2
8	C: B + 2-opt initial bound	3,386	4.0×	2.9
8	**D: C + lower bound (redesign)**	**453**	**30×**	**0.95**
10	Exhaustive (Task 6)	986,410	1×	501
10	A	66,727	14.8×	61
10	B	53,386	18.5×	52
10	C	52,358	18.8×	58
10	**D**	**3,292**	**300×**	**6.9**
12	Exhaustive (estimated e·11!, not run)	108,505,112	1×	about 56,000 (estimated)
12	A	1,007,017	108×	1,047
12	B	715,016	152×	798
12	C	695,548	156×	752
12	**D**	**27,394**	**3,960×**	**62**

Scaling of the full redesign D (10 instances per size):

n	Mean nodes	Worst of 10	Mean time (s)	Worst time (s)	Exhaustive nodes (e·(n−1)!, estimate)
10	3,292	6,544	0.006	0.010	986,410
12	27,394	137,118	0.052	0.26	1.1×10^8
14	183,177	1,210,108	0.40	2.58	1.7×10^10
16	416,868	960,678	1.02	2.24	3.6×10^12
18	2,931,597	11,187,117	7.65	28.9	9.7×10^14





Per-node cost (instance n = 10, seed 3): exhaustive 986,410 nodes in 0.376 s (2.6 million nodes/s); redesign 4,624 nodes in 7.4 ms (0.63 million nodes/s): each node is about 4 times more expensive, as predicted, and the preprocessing (2-opt bound) took 0.18 ms. Both returned the same cost.

7.6 Defence of the final design (2 marks)

- **Objective met?** (a) Yes: identical optimal costs on all 20 verified instances (n = 8, 10). (b) Yes: 300× fewer nodes at n = 10 (target 100×), and 3,960× at n = 12 against an estimated baseline. (c) Yes on average: n = 14 takes 0.40 s on average, but the worst instance of 10 took 2.58 s, over the 2 s limit, so the guarantee is only for n ⩽ 12 (worst 0.26 s). (d) Yes.
- **Prediction supported?** Predictions 1 to 3 were supported (300× nodes, 55–73× less time at n = 10 [0.376–0.50 s against 6.9 ms], per-node cost ×4). **Ablation finding (not predicted):** nearest-first ordering gave only 1.25× (66,727 → 53,386 nodes at n = 10) and the 2-opt initial bound only 1.02× (52,358), while the **partial-cost prune alone gave 14.8× and the lower bound a further 16×**. Likely reasons (not tested separately): with a random start, the first tour found by a depth-first search is already decent, so a better initial bound changes little; and a bound that only looks at the cost so far cannot see that an apparently cheap prefix still has to visit cities that are far away, which is exactly what the lower bound captures.
- **Magnitude and new costs:** 300× fewer nodes and about 55–73× less time at n = 10; a gain that grows with n (108× → 3,960× from n = 10 to 12 in nodes against the estimate). New costs: per-node work ×4, extra preprocessing (neighbour lists, 2-opt tour), instance-dependent running time with heavy tails (n = 18: mean 7.7 s, worst 28.9 s) and the worst case is still exponential.

- **When the original is still preferable:** for very small n (⩽ 6: nodes ⩽ 326) where the bookkeeping is not worth it, when the code must be trivially verifiable, and when *every* tour must be produced or counted (enumeration) rather than just the best. When the exact optimum is not needed and n > 16, the 2-opt local search of Task 6 remains the better deployment choice.
- **Deployment boundary after the redesign (2 s response time):** exact search with B&B for n ⩽ 12 (guaranteed in my tests), up to n ≈ 16 with a time-out that returns the best tour found so far (never worse than the 2-opt tour, which is already available as the initial bound), and local search beyond.

Source Code, Raw Data and How to Reproduce

Requires Python 3 with numpy and matplotlib. Run each file with `python3 <file>`; each writes its raw results to a JSON file next to it. Random seeds are fixed in the code.

Task	Source code	Raw data written	Seeds
1	`task1_search.py`	`task1_results.json`	maze 3; open maps 1 and 100–149; cluttered 5
2	`task2_online.py`	`task2_results.json`	mazes 0–9
3	`task3_alphabeta.py`	`task3_results.json`	tree seed 1
4	`task4_minimax.py`	`task4_results.json`	games 0–19, mid-game position 5
5	`task5_csp.py`	`task5_results.json`, `fig_csp_1a.png`, `fig_csp_2.png`	instances 0–29
6	`task6_tsp.py`	`task6_results.json`	points 1, 100–102, restarts 0–19
7	`task7_bnb.py` (imports `task6_tsp.py`)	`task7_results.json`	points 0–9

Honesty notes. (1) Predictions in Tasks 1, 3, 4, 6 and 7 were written before the corresponding run, and where they were wrong (cup-trap map, worst-first alpha-beta share, random-order constant) the report says so. (2) Execution times come from a single

Python 3 run on one cloud machine and are secondary evidence; operation counts are the primary evidence. (3) The exhaustive-search figures for n = 12 and above in Task 7 are estimates from the verified formula e·(n − 1)!, not runs. (4) Tests with 20 games (Task 4) are small and differences of one or two games are noise.